

Ch 04: For and While Loops

Application Program Development

Derek Harter

Department of Computer Science
East Texas A&M University

Summer 2026



EAST TEXAS A&M

Introduction to Repetition Structures

- Often have to write code that performs the same task multiple times
 - Disadvantages to duplicating code
 - Makes program large
 - Time consuming
 - May need to be corrected in many places
 - **DRY Don't Repeat Yourself**
- **Repetition structure:** makes computer repeat block of code as necessary
 - **Condition-controlled loop** uses a True / False condition to control the number of times that it repeats
 - **Count-controlled loop** repeats a specific number of times

Repetition Structures

A repetition structure causes a statement or set of statements (code block) to execute repeatedly.



The while Loop: Condition-Controlled Loops (1 of 2)

- while loop: while condition is true, do something
 - Two parts
 - 1 Condition tested to see if True or False
 - 2 Statements (code block) repeated as long as condition is True

while condition:
statement
statement
etc.

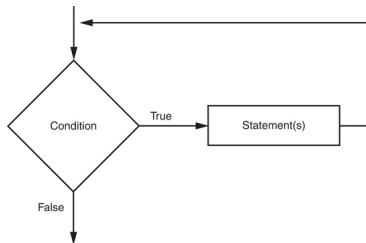
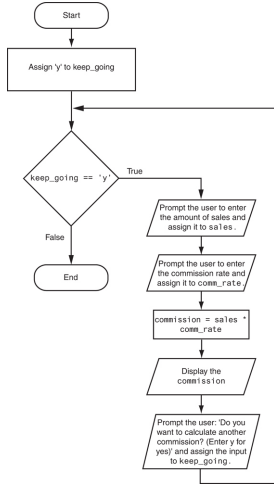


Figure 1: The logic of a while loop (Gaddis, 2021, pg.193)



The while Loop: Condition-Controlled Loops (2 of 2)



- In order for a loop to stop executing, something has to happen inside the loop to make the condition False
- **Iteration**: one execution of the body of a loop
- while loop is known as a **pretest** loop
 - Tests condition performing an iteration
 - Will never execute if condition is False at start
 - Required performing some steps prior to the loop

Figure 2: Flowchart for sales commission program (Gaddis, 2021, pg.196)



Infinite Loop

Loop that does not have a way of stopping

- Repeats until program is interrupted
- Occurs when programmer forgets to include stopping code in the loop
- **Bug**, use `ctrl-c` (usually) to interrupt running program

- In all but rare cases, loops **MUST** contain within themselves a way to terminate
- Something inside a `while` loop must eventually make the test condition `False`



The for Loop: a Count-Controlled Loop (1 of 2)

- **Count-Controlled loop:** iterates a specific number of times

- Use a for statement to write count-controlled loop
- Designed to work with a sequence of data items
 - Iterates once for each item in the sequence
- **Target variable:** the variable which is the target of the assignment at the beginning of each iteration

```
for variable in [value1, value2, etc.]:  
    statement  
    statement  
    etc.
```



The for Loop: a Count-Controlled Loop (2 of 2)

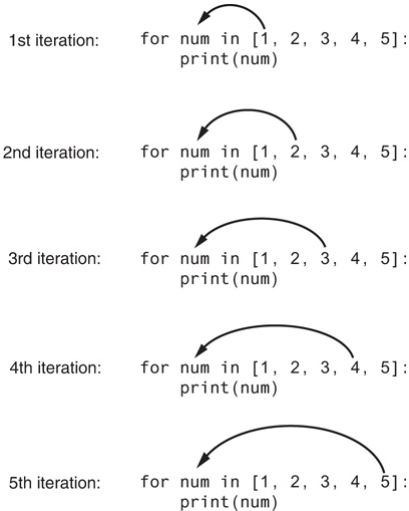


Figure 3: The for loop (Gaddis, [2021](#), pg.201)



Using the range() Function with the for Loop

- The range() function simplifies the process of writing a for loop
 - range() returns an iterable object
 - **Iterable:** contains a sequence of values that can be iterated over
- range() characteristics
 - One argument: used as ending limit
 - Two arguments: starting value and ending limit
 - Three arguments: third argument is step value

```
>>> for idx in range(10):
...     print(idx, end=',')
...
0,1,2,3,4,5,6,7,8,9,

>>> for idx in range(5,10):
...     print(idx, end=',')
...
5,6,7,8,9,

>>> for idx in range(2, 10, 2):
...     print(idx, end=',')
...
2,4,6,8,>>>

>>> for idx in range(10, 1, -1):
...     print(idx, end=',')
...
10,9,8,7,6,5,4,3,2,
```



Using the Target Variable Inside the Loop

- Purpose of target variable is to reference each item in a sequence as the loop iterates
- Target variable can be used in calculations or tasks in the body of the loop
 - Example: calculate square root of each number in a range

```
>>> for number in range(1, 11):  
...     square = number**2  
...     print(f'{number:5d}{square:5d}')  
...  
1      1  
2      4  
3      9  
4     16  
5     25  
6     36  
7     49  
8     64  
9     81  
10    100
```



Summary

- Repetition structures allow you to write code that performs a block of statements multiple times
 - Avoids code duplication
 - Used to iterate over items in a list or array
- `while` loops typically used for **condition-controlled loops**, where they repeat until some condition becomes `False`
- `for` loops typically used for **count-controlled loops** that iterate a specific number of times, e.g. one time for each item in a list of items
- Use the Python built-in `range()` function to generate explicit iterable list of integers for a count-controlled loop



References

Gaddis, T. (2021). *Starting out with python* (5th). Pearson.



EAST TEXAS A&M